```
import pandas as pd
```

```
df=pd.read_csv("/content/Salary_dataset.csv")
df
```

	Unnamed: 0	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0
5	5	3.0	56643.0
6	6	3.1	60151.0
7	7	3.3	54446.0
8	8	3.3	64446.0
9	9	3.8	57190.0
10	10	4.0	63219.0
11	11	4.1	55795.0
12	12	4.1	56958.0
13	13	4.2	57082.0
14	14	4.6	61112.0
15	15	5.0	67939.0
16	16	5.2	66030.0
17	17	5.4	83089.0
18	18	6.0	81364.0
19	19	6.1	93941.0
20	20	6.9	91739.0
21	21	7.2	98274.0
22	22	8.0	101303.0
23	23	8.3	113813.0
24	24	8.8	109432.0
25	25	9.1	105583.0
26	26	9.6	116970.0
27	27	9.7	112636.0
28	28	10.4	122392.0
29	29	10.6	121873.0

```
df.head()
```

	Unnamed: 0	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0

```
df.tail()
```

	Unnamed: 0	YearsExperience	Salary
25	25	9.1	105583.0
26	26	9.6	116970.0
27	27	9.7	112636.0
28	28	10.4	122392.0
29	29	10.6	121873.0

```
x=df.YearsExperience
y=df.Salary
x
y
```

	Salary
0	39344.0
1	46206.0
2	37732.0
3	43526.0
4	39892.0
5	56643.0
6	60151.0
7	54446.0
8	64446.0
9	57190.0
10	63219.0
11	55795.0
12	56958.0
13	57082.0
14	61112.0
15	67939.0
16	66030.0
17	83089.0
18	81364.0
19	93941.0
20	91739.0
21	98274.0
22	101303.0
23	113813.0
24	109432.0
25	105583.0
26	116970.0
27	112636.0
28	122392.0
29	121873.0

dtype: float64

```
x=df["YearsExperience"].values
y=df["Salary"].values
x
```

```
array([ 1.2,  1.4,  1.6,  2.1,  2.3,  3. ,  3.1,  3.3,  3.3,  3.8,  4. ,
        4.1,  4.1,  4.2,  4.6,  5. ,  5.2,  5.4,  6. ,  6.1,  6.9,  7.2,
        8. ,  8.3,  8.8,  9.1,  9.6,  9.7, 10.4, 10.6])
```

```
Start coding or generate with AI.
```

```
x=df.iloc[:,1].values
```

```
x
```

```
array([ 1.2,  1.4,  1.6,  2.1,  2.3,  3. ,  3.1,  3.3,  3.3,  3.8,  4. ,
        4.1,  4.1,  4.2,  4.6,  5. ,  5.2,  5.4,  6. ,  6.1,  6.9,  7.2,
        8. ,  8.3,  8.8,  9.1,  9.6,  9.7, 10.4, 10.6])
```

```
y=df.iloc[:,2].values
y
```

```
array([ 39344.,  46206.,  37732.,  43526.,  39892.,  56643.,  60151.,
        54446.,  64446.,  57190.,  63219.,  55795.,  56958.,  57082.,
        61112.,  67939.,  66030.,  83089.,  81364.,  93941.,  91739.,
        98274., 101303., 113813., 109432., 105583., 116970., 112636.,
       122392., 121873.])
```

```
y=df.iloc[:,-2].values
y
```

```
array([ 1.2,  1.4,  1.6,  2.1,  2.3,  3. ,  3.1,  3.3,  3.3,  3.8,  4. ,
        4.1,  4.1,  4.2,  4.6,  5. ,  5.2,  5.4,  6. ,  6.1,  6.9,  7.2,
        8. ,  8.3,  8.8,  9.1,  9.6,  9.7, 10.4, 10.6])
```

```
import numpy as np
X=np.array(x)
Y=np.array(y)
X
```

```
array([ 1.2,  1.4,  1.6,  2.1,  2.3,  3. ,  3.1,  3.3,  3.3,  3.8,  4. ,
        4.1,  4.1,  4.2,  4.6,  5. ,  5.2,  5.4,  6. ,  6.1,  6.9,  7.2,
        8. ,  8.3,  8.8,  9.1,  9.6,  9.7, 10.4, 10.6])
```

```
X=X.reshape(-1,1)
X
```

```
array([[ 1.2],
       [ 1.4],
       [ 1.6],
       [ 2.1],
       [ 2.3],
       [ 3. ],
       [ 3.1],
       [ 3.3],
       [ 3.3],
       [ 3.8],
       [ 4. ],
       [ 4.1],
       [ 4.1],
       [ 4.2],
       [ 4.6],
       [ 5. ],
       [ 5.2],
       [ 5.4],
       [ 6. ],
       [ 6.1],
       [ 6.9],
       [ 7.2],
       [ 8. ],
       [ 8.3],
       [ 8.8],
       [ 9.1],
       [ 9.6],
       [ 9.7],
       [10.4],
       [10.6]])
```

```
X.max()
```

```
np.float64(10.6)
```

```
X.min()
```

```
np.float64(1.2000000000000002)
```

```
X.mean()
```

```
np.float64(5.413333333333332)
```

```
X.std()
```

```
np.float64(2.790189161249745)
```

```
x_standard=(X-X.mean())/X.std()
x_standard
```

```
array([[-1.51005294],
       [-1.43837321],
       [-1.36669348],
       [-1.18749416],
       [-1.11581443],
       [-0.86493538],
       [-0.82909552],
       [-0.75741579],
       [-0.75741579],
       [-0.57821647],
       [-0.50653674],
       [-0.47069688],
       [-0.47069688],
       [-0.43485702],
       [-0.29149756],
       [-0.1481381 ],
       [-0.07645838],
       [-0.00477865],
       [ 0.21026054],
       [ 0.2461004 ],
       [ 0.53281931],
       [ 0.6403389 ],
       [ 0.92705781],
       [ 1.03457741],
       [ 1.21377673],
       [ 1.32129632],
       [ 1.50049564],
       [ 1.5363355 ],
       [ 1.78721455],
       [ 1.85889428]])
```

```
x_min=X.min()
x_max=X.max()
x_n=(X-x_min)/(x_max-x_min)
x_n
```

```
array([[0.        ],
       [0.0212766 ],
       [0.04255319],
       [0.09574468],
       [0.11702128],
       [0.19148936],
       [0.20212766],
       [0.22340426],
       [0.22340426],
       [0.27659574],
       [0.29787234],
       [0.30851064],
       [0.30851064],
       [0.31914894],
       [0.36170213],
       [0.40425532],
       [0.42553191],
       [0.44680851],
       [0.5106383 ],
       [0.5212766 ],
       [0.60638298],
       [0.63829787],
       [0.72340426],
       [0.75531915],
       [0.80851064],
       [0.84042553],
       [0.89361702],
       [0.90425532],
       [0.9787234 ],
       [1.        ]])
```

```
m=np.random.randn()
c=np.random.randn()
print("randomly initialized slope",m)
print("randomly initialized intercept",c)
```

```
randomly initialized slope 2.202414256736805
randomly initialized intercept 1.107765037650964
```

```
def predict(X,m,c):
  y_pred=m*X+c
  return y_pred
```

```
y_pred=predict(x_n,m,c)
```

```
print(y_pred[:5])
```

```
[[1.10776504]
 [1.15462492]
 [1.20148479]
 [1.31863449]
 [1.36549437]]
```

```
def compute_cost(y,y_pred):
  m=len(y)
  cost=(1/(2*m))*np.sum((y_pred-Y)**2)
  return cost
```

```
cost=compute_cost(y,y_pred)
print("cost",cost)
```

```
cost 288.36333689228985
```

```
m=0.0
c=0.0
y_pred=predict(x_n,m,c)
cost=compute_cost(y,y_pred)
print("cost",cost)
```

```
cost 556.3399999999999
```

```
learning_rate=0.01
epochs=1000
print(learning_rate)
print(epochs)
```

```
0.01
1000
```

```
cost_history=[]
```

```
n=len(y)
```

```
for epoch in range(epochs):
  y_pred=predict(x_n,m,c)
  dm=(1/n)*np.sum((y_pred-y)*x_n)
  dc=(1/n)*np.sum(y_pred-y)
  m=m-learning_rate*dm
  c=c-learning_rate*dc
  cost=compute_cost(y,y_pred)
  cost_history.append(cost)
  if epoch%100==0:
    print("epoch",epoch,"cost",cost)
```

```
epoch 0 cost 556.3399999999999
epoch 100 cost 116.85182524722308
epoch 200 cost 116.7782541592273
epoch 300 cost 116.77734471604968
epoch 400 cost 116.77733347403986
epoch 500 cost 116.77733333507264
epoch 600 cost 116.77733333335482
epoch 700 cost 116.77733333333357
epoch 800 cost 116.77733333333332
epoch 900 cost 116.77733333333332
```

```
final_slope=m
final_intercept=c
print("final slope",final_slope)
print("final intercept",final_intercept)
```

```
final slope 6.120870437955232e-10
final intercept 5.413333333037559
```

```
def predict_salary(YearsExperience,m,c,x_min,x_max):
  x=np.array(YearsExperience)
  x_n=(x-x_min)/(x_max-x_min)
  sal_pred=m*x_n+c
  return sal_pred
  predict_salary
```

```
print(predict_salary)
```

```
<function predict_salary at 0x7fd0a140d8a0>
```

```
years=5
predicted_salary=predict_salary(years,final_slope,final_intercept,x_min,x_max)
print(f"salary for {years} yr of experience")
print(predicted_salary)
```

```
salary for 5 yr of experience
5.413333333284998
```